

```
In [144... import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
In [145... import warnings
warnings.filterwarnings('ignore')
```

```
In [146... df = pd.read_csv(r"C:\Users\santo\OneDrive\Desktop\Data science\Jan 2026\6th - Navie Bayes\project\adult.csv")
```

```
In [147... df.shape
```

Out[147... (32560, 15)

```
In [148... df.head()
```

Out[148...

	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States	<=50K
0	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13	United-States	<=50K
1	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United-States	<=50K
2	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	40	United-States	<=50K
3	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	40	Cuba	<=50K
4	37	Private	284582	Masters	14	Married-civ-spouse	Exec-managerial	Wife	White	Female	0	0	40	United-States	<=50K

```
In [149... col_names = ['age', 'workclass', 'fnlwgt', 'education', 'education_num', 'marital_status', 'occupation', 'relationship',
               'race', 'sex', 'capital_gain', 'capital_loss', 'hours_per_week', 'native_country', 'income']
```

```
df.columns = col_names
```

```
df.columns
```

```
Out[149...] Index(['age', 'workclass', 'fnlwgt', 'education', 'education_num',
      'marital_status', 'occupation', 'relationship', 'race', 'sex',
      'capital_gain', 'capital_loss', 'hours_per_week', 'native_country',
      'income'],
      dtype='object')
```

```
In [150...] df.head()
```

```
Out[150...]
   age  workclass  fnlwgt  education  education_num  marital_status  occupation  relationship  race  sex  capital_gain  capital_loss  |
0   50  Self-emp-  83311  Bachelors           13  Married-civ-  Exec-  Husband  White  Male           0           0
   not-inc
1   38   Private  215646   HS-grad            9   Divorced  Handlers-  Not-in-  White  Male           0           0
   cleaners  family
2   53   Private  234721    11th             7  Married-civ-  Handlers-  Husband  Black  Male           0           0
   cleaners
3   28   Private  338409  Bachelors           13  Married-civ-  Prof-  Wife  Black  Female           0           0
   specialty
4   37   Private  284582   Masters           14  Married-civ-  Exec-  Wife  White  Female           0           0
   managerial
```



```
In [151...] df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32560 entries, 0 to 32559
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   age                   32560 non-null  int64
1   workclass             32560 non-null  object
2   fnlwgt               32560 non-null  int64
3   education            32560 non-null  object
4   education_num        32560 non-null  int64
5   marital_status       32560 non-null  object
6   occupation           32560 non-null  object
7   relationship         32560 non-null  object
8   race                 32560 non-null  object
9   sex                 32560 non-null  object
10  capital_gain         32560 non-null  int64
11  capital_loss         32560 non-null  int64
12  hours_per_week       32560 non-null  int64
13  native_country       32560 non-null  object
14  income              32560 non-null  object
dtypes: int64(6), object(9)
memory usage: 3.7+ MB
```

Explore Categorical variables

```
In [152... categorical = [var for var in df.columns if df[var].dtype=='O']

print('There are {} categorical variables\n'.format(len(categorical)))

print('The categorical variables are :\n\n', categorical)
```

There are 9 categorical variables

The categorical variables are :

```
['workclass', 'education', 'marital_status', 'occupation', 'relationship', 'race', 'sex', 'native_country', 'income']
```

```
In [153... df[categorical].head()
```

Out[153...

	workclass	education	marital_status	occupation	relationship	race	sex	native_country	income
0	Self-emp-not-inc	Bachelors	Married-civ-spouse	Exec-managerial	Husband	White	Male	United-States	<=50K
1	Private	HS-grad	Divorced	Handlers-cleaners	Not-in-family	White	Male	United-States	<=50K
2	Private	11th	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	United-States	<=50K
3	Private	Bachelors	Married-civ-spouse	Prof-specialty	Wife	Black	Female	Cuba	<=50K
4	Private	Masters	Married-civ-spouse	Exec-managerial	Wife	White	Female	United-States	<=50K

In [154...

```
df[categorical].isnull().sum()
```

Out[154...

```
workclass      0
education      0
marital_status 0
occupation     0
relationship    0
race           0
sex            0
native_country 0
income         0
dtype: int64
```

In [155...

```
for var in categorical:
    print(df[var].value_counts())
```

```
workclass
Private      22696
Self-emp-not-inc  2541
Local-gov    2093
?            1836
State-gov    1297
Self-emp-inc 1116
Federal-gov   960
Without-pay   14
Never-worked   7
Name: count, dtype: int64
education
HS-grad      10501
Some-college  7291
Bachelors    5354
Masters       1723
Assoc-voc    1382
11th          1175
Assoc-acdm    1067
10th          933
7th-8th       646
Prof-school   576
9th           514
12th          433
Doctorate     413
5th-6th       333
1st-4th       168
Preschool     51
Name: count, dtype: int64
marital_status
Married-civ-spouse  14976
Never-married       10682
Divorced             4443
Separated            1025
Widowed              993
Married-spouse-absent  418
Married-AF-spouse      23
Name: count, dtype: int64
occupation
Prof-specialty  4140
Craft-repair    4099
```

Exec-managerial	4066
Adm-clerical	3769
Sales	3650
Other-service	3295
Machine-op-inspct	2002
?	1843
Transport-moving	1597
Handlers-cleaners	1370
Farming-fishing	994
Tech-support	928
Protective-serv	649
Priv-house-serv	149
Armed-Forces	9

Name: count, dtype: int64

relationship

Husband	13193
Not-in-family	8304
Own-child	5068
Unmarried	3446
Wife	1568
Other-relative	981

Name: count, dtype: int64

race

White	27815
Black	3124
Asian-Pac-Islander	1039
Amer-Indian-Eskimo	311
Other	271

Name: count, dtype: int64

sex

Male	21789
Female	10771

Name: count, dtype: int64

native_country

United-States	29169
Mexico	643
?	583
Philippines	198
Germany	137
Canada	121
Puerto-Rico	114

El-Salvador	106
India	100
Cuba	95
England	90
Jamaica	81
South	80
China	75
Italy	73
Dominican-Republic	70
Vietnam	67
Guatemala	64
Japan	62
Poland	60
Columbia	59
Taiwan	51
Haiti	44
Iran	43
Portugal	37
Nicaragua	34
Peru	31
France	29
Greece	29
Ecuador	28
Ireland	24
Hong	20
Trinidad&Tobago	19
Cambodia	19
Thailand	18
Laos	18
Yugoslavia	16
Outlying-US(Guam-USVI-etc)	14
Honduras	13
Hungary	13
Scotland	12
Holand-Netherlands	1

Name: count, dtype: int64

income

<=50K	24719
>50K	7841

Name: count, dtype: int64

```
In [156... for var in categorical:  
    print(df[var].value_counts()/np.float64(len(df)))
```

```
workclass
Private      0.697052
Self-emp-not-inc  0.078041
Local-gov    0.064281
?            0.056388
State-gov    0.039834
Self-emp-inc  0.034275
Federal-gov  0.029484
Without-pay  0.000430
Never-worked 0.000215
Name: count, dtype: float64
education
HS-grad      0.322512
Some-college 0.223925
Bachelors    0.164435
Masters      0.052918
Assoc-voc    0.042445
11th         0.036087
Assoc-acdm   0.032770
10th         0.028655
7th-8th      0.019840
Prof-school  0.017690
9th          0.015786
12th         0.013299
Doctorate    0.012684
5th-6th      0.010227
1st-4th      0.005160
Preschool    0.001566
Name: count, dtype: float64
marital_status
Married-civ-spouse  0.459951
Never-married       0.328071
Divorced            0.136456
Separated           0.031480
Widowed            0.030498
Married-spouse-absent 0.012838
Married-AF-spouse    0.000706
Name: count, dtype: float64
occupation
Prof-specialty  0.127150
Craft-repair    0.125891
```

```

Exec-managerial      0.124877
Adm-clerical         0.115756
Sales                0.112101
Other-service        0.101198
Machine-op-inspct    0.061486
?                   0.056603
Transport-moving     0.049048
Handlers-cleaners    0.042076
Farming-fishing      0.030528
Tech-support         0.028501
Protective-serv      0.019932
Priv-house-serv      0.004576
Armed-Forces         0.000276
Name: count, dtype: float64
relationship
Husband              0.405190
Not-in-family        0.255037
Own-child             0.155651
Unmarried            0.105835
Wife                 0.048157
Other-relative       0.030129
Name: count, dtype: float64
race
White                0.854269
Black                0.095946
Asian-Pac-Islander   0.031910
Amer-Indian-Eskimo   0.009552
Other                0.008323
Name: count, dtype: float64
sex
Male                0.669195
Female              0.330805
Name: count, dtype: float64
native_country
United-States        0.895854
Mexico               0.019748
?                   0.017905
Philippines          0.006081
Germany              0.004208
Canada               0.003716
Puerto-Rico         0.003501

```

El-Salvador	0.003256
India	0.003071
Cuba	0.002918
England	0.002764
Jamaica	0.002488
South	0.002457
China	0.002303
Italy	0.002242
Dominican-Republic	0.002150
Vietnam	0.002058
Guatemala	0.001966
Japan	0.001904
Poland	0.001843
Columbia	0.001812
Taiwan	0.001566
Haiti	0.001351
Iran	0.001321
Portugal	0.001136
Nicaragua	0.001044
Peru	0.000952
France	0.000891
Greece	0.000891
Ecuador	0.000860
Ireland	0.000737
Hong	0.000614
Trinidad&Tobago	0.000584
Cambodia	0.000584
Thailand	0.000553
Laos	0.000553
Yugoslavia	0.000491
Outlying-US(Guam-USVI-etc)	0.000430
Honduras	0.000399
Hungary	0.000399
Scotland	0.000369
Holand-Netherlands	0.000031

Name: count, dtype: float64

income

<=50K 0.759183

>50K 0.240817

Name: count, dtype: float64

```
In [157... df.workclass.unique()
```

```
Out[157... array([' Self-emp-not-inc', ' Private', ' State-gov', ' Federal-gov',  
      ' Local-gov', ' ?', ' Self-emp-inc', ' Without-pay',  
      ' Never-worked'], dtype=object)
```

```
In [158... df.workclass.value_counts()
```

```
Out[158... workclass  
Private          22696  
Self-emp-not-inc 2541  
Local-gov        2093  
?                1836  
State-gov        1297  
Self-emp-inc     1116  
Federal-gov      960  
Without-pay      14  
Never-worked     7  
Name: count, dtype: int64
```

```
In [159... df['workclass'].replace('?', np.nan, inplace=True)
```

```
In [160... df.workclass.value_counts()
```

```
Out[160... workclass  
Private          22696  
Self-emp-not-inc 2541  
Local-gov        2093  
?                1836  
State-gov        1297  
Self-emp-inc     1116  
Federal-gov      960  
Without-pay      14  
Never-worked     7  
Name: count, dtype: int64
```

```
In [161... df.occupation.unique()
```

```
Out[161...] array([' Exec-managerial', ' Handlers-cleaners', ' Prof-specialty',  
      ' Other-service', ' Adm-clerical', ' Sales', ' Craft-repair',  
      ' Transport-moving', ' Farming-fishing', ' Machine-op-inspct',  
      ' Tech-support', ' ?', ' Protective-serv', ' Armed-Forces',  
      ' Priv-house-serv'], dtype=object)
```

```
In [162...] df.occupation.value_counts()
```

```
Out[162...] occupation  
Prof-specialty      4140  
Craft-repair        4099  
Exec-managerial     4066  
Adm-clerical        3769  
Sales               3650  
Other-service       3295  
Machine-op-inspct   2002  
?                   1843  
Transport-moving    1597  
Handlers-cleaners   1370  
Farming-fishing     994  
Tech-support        928  
Protective-serv     649  
Priv-house-serv     149  
Armed-Forces         9  
Name: count, dtype: int64
```

```
In [163...] df['occupation'].replace('?', np.nan, inplace=True)
```

```
In [164...] df.occupation.value_counts()
```

```
Out[164... occupation
Prof-specialty      4140
Craft-repair        4099
Exec-managerial     4066
Adm-clerical        3769
Sales               3650
Other-service       3295
Machine-op-inspct   2002
?                  1843
Transport-moving    1597
Handlers-cleaners   1370
Farming-fishing     994
Tech-support        928
Protective-serv     649
Priv-house-serv     149
Armed-Forces        9
Name: count, dtype: int64
```

```
In [165... df.native_country.unique()
```

```
Out[165... array([' United-States', ' Cuba', ' Jamaica', ' India', ' ?', ' Mexico',
      ' South', ' Puerto-Rico', ' Honduras', ' England', ' Canada',
      ' Germany', ' Iran', ' Philippines', ' Italy', ' Poland',
      ' Columbia', ' Cambodia', ' Thailand', ' Ecuador', ' Laos',
      ' Taiwan', ' Haiti', ' Portugal', ' Dominican-Republic',
      ' El-Salvador', ' France', ' Guatemala', ' China', ' Japan',
      ' Yugoslavia', ' Peru', ' Outlying-US(Guam-USVI-etc)', ' Scotland',
      ' Trinidad&Tobago', ' Greece', ' Nicaragua', ' Vietnam', ' Hong',
      ' Ireland', ' Hungary', ' Holand-Netherlands'], dtype=object)
```

```
In [166... df.native_country.value_counts()
```

```
Out[166... native_country
United-States 29169
Mexico 643
? 583
Philippines 198
Germany 137
Canada 121
Puerto-Rico 114
El-Salvador 106
India 100
Cuba 95
England 90
Jamaica 81
South 80
China 75
Italy 73
Dominican-Republic 70
Vietnam 67
Guatemala 64
Japan 62
Poland 60
Columbia 59
Taiwan 51
Haiti 44
Iran 43
Portugal 37
Nicaragua 34
Peru 31
France 29
Greece 29
Ecuador 28
Ireland 24
Hong 20
Trinidad&Tobago 19
Cambodia 19
Thailand 18
Laos 18
Yugoslavia 16
Outlying-US(Guam-USVI-etc) 14
Honduras 13
```

```
Hungary          13
Scotland         12
Holand-Netherlands 1
Name: count, dtype: int64
```

```
In [167... df[categorical].isnull().sum()
```

```
Out[167... workclass      0
education    0
marital_status 0
occupation   0
relationship  0
race         0
sex          0
native_country 0
income       0
dtype: int64
```

```
In [168... for var in categorical:
    print(var, 'contains', len(df[var].unique()), 'labels')
```

```
workclass contains 9 labels
education contains 16 labels
marital_status contains 7 labels
occupation contains 15 labels
relationship contains 6 labels
race contains 5 labels
sex contains 2 labels
native_country contains 42 labels
income contains 2 labels
```

```
In [169... numerical = [var for var in df.columns if df[var].dtype!='O']

print('There are {} numerical variables\n'.format(len(numerical)))

print('The numerical variables are :', numerical)
```

There are 6 numerical variables

The numerical variables are : ['age', 'fnlwgt', 'education_num', 'capital_gain', 'capital_loss', 'hours_per_week']

In [170...] `df[numerical].head()`

Out[170...]

	age	fnlwgt	education_num	capital_gain	capital_loss	hours_per_week
0	50	83311	13	0	0	13
1	38	215646	9	0	0	40
2	53	234721	7	0	0	40
3	28	338409	13	0	0	40
4	37	284582	14	0	0	40

In [171...] `df[numerical].isnull().sum()`

Out[171...]

age	0
fnlwgt	0
education_num	0
capital_gain	0
capital_loss	0
hours_per_week	0

dtype: int64

In [172...] `X = df.drop(['income'],axis=1)`
`y = df['income']`

In [173...] `# split X and y into training and testing sets`

`from sklearn.model_selection import train_test_split`

`X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3, random_state = 0)`

In [174...] `X_train.shape, X_test.shape`

Out[174...] `((22792, 14), (9768, 14))`

In [175...] `X_train.dtypes`

```
Out[175...  age          int64
workclass     object
fnlwgt        int64
education     object
education_num int64
marital_status object
occupation    object
relationship   object
race          object
sex           object
capital_gain  int64
capital_loss  int64
hours_per_week int64
native_country object
dtype: object
```

```
In [176... # display categorical variables

categorical = [col for col in X_train.columns if X_train[col].dtypes == 'O']

categorical
```

```
Out[176... ['workclass',
'education',
'marital_status',
'occupation',
'relationship',
'race',
'sex',
'native_country']
```

```
In [177... # display numerical variables

numerical = [col for col in X_train.columns if X_train[col].dtypes != 'O']

numerical
```

```
Out[177... ['age',  
            'fnlwgt',  
            'education_num',  
            'capital_gain',  
            'capital_loss',  
            'hours_per_week']
```

```
In [178... # print percentage of missing values in the categorical variables in training set
```

```
X_train[categorical].isnull().mean()
```

```
Out[178... workclass      0.0  
education      0.0  
marital_status 0.0  
occupation     0.0  
relationship   0.0  
race           0.0  
sex            0.0  
native_country 0.0  
dtype: float64
```

```
In [179... for col in categorical:  
    if X_train[col].isnull().mean()>0:  
        print(col, (X_train[col].isnull().mean()))
```

```
In [180... # impute missing categorical variables with most frequent value
```

```
for df2 in [X_train, X_test]:  
    df2['workclass'].fillna(X_train['workclass'].mode()[0], inplace=True)  
    df2['occupation'].fillna(X_train['occupation'].mode()[0], inplace=True)  
    df2['native_country'].fillna(X_train['native_country'].mode()[0], inplace=True)
```

```
In [181... # check missing values in categorical variables in X_train
```

```
X_train[categorical].isnull().sum()
```

```
Out[181...] workclass      0
            education    0
            marital_status 0
            occupation    0
            relationship   0
            race           0
            sex            0
            native_country 0
            dtype: int64
```

```
In [182...] X_test[categorical].isnull().sum()
```

```
Out[182...] workclass      0
            education    0
            marital_status 0
            occupation    0
            relationship   0
            race           0
            sex            0
            native_country 0
            dtype: int64
```

```
In [183...] X_train.isnull().sum()
```

```
Out[183...] age           0
            workclass      0
            fnlwgt         0
            education      0
            education_num   0
            marital_status  0
            occupation      0
            relationship     0
            race            0
            sex             0
            capital_gain     0
            capital_loss     0
            hours_per_week   0
            native_country   0
            dtype: int64
```

```
In [184... X_test.isnull().sum()
```

```
Out[184... age                0
workclass            0
fnlwgt              0
education            0
education_num        0
marital_status       0
occupation           0
relationship         0
race                 0
sex                  0
capital_gain         0
capital_loss         0
hours_per_week       0
native_country       0
dtype: int64
```

```
In [185... categorical
```

```
Out[185... ['workclass',
'education',
'marital_status',
'occupation',
'relationship',
'race',
'sex',
'native_country']
```

```
In [186... X_train[categorical].head()
```

Out[186...

	workclass	education	marital_status	occupation	relationship	race	sex	native_country
20721	Self-emp-inc	Some-college	Married-civ-spouse	Transport-moving	Husband	Black	Male	Haiti
32097	Private	HS-grad	Married-civ-spouse	Craft-repair	Husband	White	Male	United-States
25205	State-gov	HS-grad	Divorced	Adm-clerical	Unmarried	White	Female	United-States
23491	Private	Bachelors	Never-married	Farming-fishing	Not-in-family	White	Male	United-States
12367	Private	Some-college	Never-married	Adm-clerical	Own-child	White	Male	India

In [187...

!pip install category_encoders

Requirement already satisfied: category_encoders in c:\users\santo\anaconda3\lib\site-packages (2.9.0)
 Requirement already satisfied: numpy>=1.14.0 in c:\users\santo\anaconda3\lib\site-packages (from category_encoders) (2.1.3)
 Requirement already satisfied: pandas>=1.0.5 in c:\users\santo\anaconda3\lib\site-packages (from category_encoders) (2.2.3)
 Requirement already satisfied: patsy>=0.5.1 in c:\users\santo\anaconda3\lib\site-packages (from category_encoders) (1.0.1)
 Requirement already satisfied: scikit-learn>=1.6.0 in c:\users\santo\anaconda3\lib\site-packages (from category_encoders) (1.6.1)
 Requirement already satisfied: scipy>=1.0.0 in c:\users\santo\anaconda3\lib\site-packages (from category_encoders) (1.15.3)
 Requirement already satisfied: statsmodels>=0.9.0 in c:\users\santo\anaconda3\lib\site-packages (from category_encoders) (0.14.4)
 Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\santo\anaconda3\lib\site-packages (from pandas>=1.0.5->category_encoders) (2.9.0.post0)
 Requirement already satisfied: pytz>=2020.1 in c:\users\santo\anaconda3\lib\site-packages (from pandas>=1.0.5->category_encoders) (2024.1)
 Requirement already satisfied: tzdata>=2022.7 in c:\users\santo\anaconda3\lib\site-packages (from pandas>=1.0.5->category_encoders) (2025.2)
 Requirement already satisfied: six>=1.5 in c:\users\santo\anaconda3\lib\site-packages (from python-dateutil>=2.8.2->pandas>=1.0.5->category_encoders) (1.17.0)
 Requirement already satisfied: joblib>=1.2.0 in c:\users\santo\anaconda3\lib\site-packages (from scikit-learn>=1.6.0->category_encoders) (1.4.2)
 Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\santo\anaconda3\lib\site-packages (from scikit-learn>=1.6.0->category_encoders) (3.5.0)
 Requirement already satisfied: packaging>=21.3 in c:\users\santo\anaconda3\lib\site-packages (from statsmodels>=0.9.0->category_encoders) (24.2)

In [188...

import category_encoders as ce

```
In [189... encoder = ce.OneHotEncoder(cols=['workclass', 'education', 'marital_status', 'occupation', 'relationship',  
                                'race', 'sex', 'native_country'])  
  
X_train = encoder.fit_transform(X_train)  
  
X_test = encoder.transform(X_test)
```

```
In [190... X_train.head()
```

```
Out[190...      age  workclass_1  workclass_2  workclass_3  workclass_4  workclass_5  workclass_6  workclass_7  workclass_8  workclass_9  ...  nati
```

	age	workclass_1	workclass_2	workclass_3	workclass_4	workclass_5	workclass_6	workclass_7	workclass_8	workclass_9	...	nati
20721	32	1	0	0	0	0	0	0	0	0	...	
32097	45	0	1	0	0	0	0	0	0	0	...	
25205	47	0	0	1	0	0	0	0	0	0	...	
23491	37	0	1	0	0	0	0	0	0	0	...	
12367	24	0	1	0	0	0	0	0	0	0	...	

5 rows × 108 columns



```
In [191... X_train.shape
```

```
Out[191... (22792, 108)
```

```
In [192... X_test.head()
```

Out[192...]

	age	workclass_1	workclass_2	workclass_3	workclass_4	workclass_5	workclass_6	workclass_7	workclass_8	workclass_9	...	nativ
22278	40	1	0	0	0	0	0	0	0	0	...	
8950	46	0	1	0	0	0	0	0	0	0	...	
7838	33	0	0	0	0	0	0	1	0	0	...	
16505	21	0	1	0	0	0	0	0	0	0	...	
19140	59	0	1	0	0	0	0	0	0	0	...	

5 rows × 108 columns

In [193... `X_test.shape`Out[193... `(9768, 108)`In [194... `cols = X_train.columns`

```
In [195... from sklearn.preprocessing import RobustScaler
scaler = RobustScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
```

In [196... `X_train = pd.DataFrame(X_train, columns=[cols])`In [197... `X_test = pd.DataFrame(X_test, columns=[cols])`In [198... `X_train.head()`

Out[198...

	age	workclass_1	workclass_2	workclass_3	workclass_4	workclass_5	workclass_6	workclass_7	workclass_8	workclass_9	...	native_c
0	-0.25	1.0	-1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	
1	0.40	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	
2	0.50	0.0	-1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	
3	0.00	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	
4	-0.65	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	

5 rows × 108 columns



In [199...

```

from sklearn.naive_bayes import GaussianNB

gnb = GaussianNB()

gnb.fit(X_train, y_train)

```

Out[199...

▼ GaussianNB ⓘ ?

GaussianNB()

In [200...

```

y_pred = gnb.predict(X_test)
y_pred

```

Out[200...

```

array(['>50K', '<=50K', '<=50K', ..., '>50K', '<=50K', '<=50K'],
      dtype='<U6')

```

In [201...

```

from sklearn.metrics import accuracy_score

print('Model accuracy score: {0:0.4f}'.format(accuracy_score(y_test, y_pred)))

```

Model accuracy score: 0.8062

In [202...

```

y_pred_train = gnb.predict(X_train)

```

```
y_pred_train
```

```
Out[202...] array([' <=50K', ' >50K', ' <=50K', ..., ' <=50K', ' <=50K', ' <=50K'],  
      dtype='<U6')
```

```
In [203...] print('Training-set accuracy score: {0:0.4f}'. format(accuracy_score(y_train, y_pred_train)))
```

Training-set accuracy score: 0.8021

```
In [204...] # print the scores on training and test set  
  
print('Training set score: {:.4f}'.format(gnb.score(X_train, y_train)))  
  
print('Test set score: {:.4f}'.format(gnb.score(X_test, y_test)))
```

Training set score: 0.8021

Test set score: 0.8062

```
In [205...] y_test.value_counts()
```

```
Out[205...] income  
<=50K    7454  
>50K     2314  
Name: count, dtype: int64
```

```
In [206...] null_accuracy = (7407/(7407+2362))  
  
print('Null accuracy score: {0:0.4f}'. format(null_accuracy))
```

Null accuracy score: 0.7582

```
In [207...] # Print the Confusion Matrix and slice it into four pieces  
  
from sklearn.metrics import confusion_matrix  
  
cm = confusion_matrix(y_test, y_pred)  
  
print('Confusion matrix\n\n', cm)  
  
print('\nTrue Positives(TP) = ', cm[0,0])  
  
print('\nTrue Negatives(TN) = ', cm[1,1])
```

```
print('\nFalse Positives(FP) = ', cm[0,1])  
  
print('\nFalse Negatives(FN) = ', cm[1,0])
```

Confusion matrix

```
[[5970 1484]  
 [ 409 1905]]
```

True Positives(TP) = 5970

True Negatives(TN) = 1905

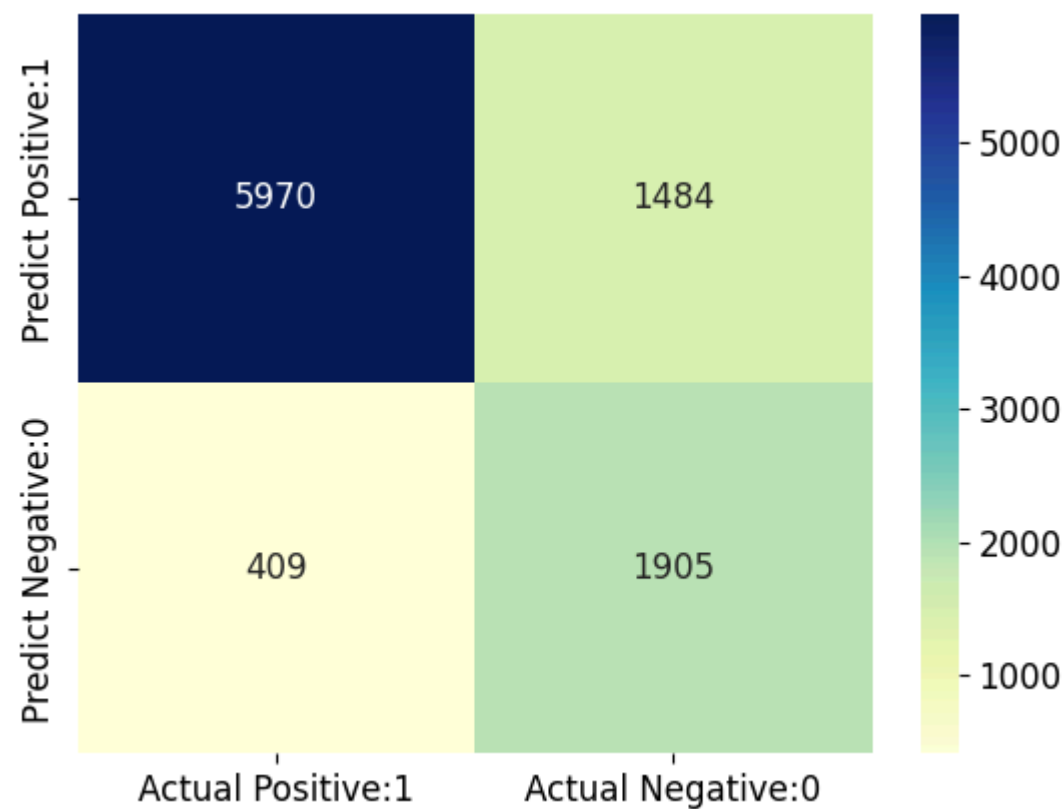
False Positives(FP) = 1484

False Negatives(FN) = 409

```
In [208... # visualize confusion matrix with seaborn heatmap  
  
cm_matrix = pd.DataFrame(data=cm, columns=['Actual Positive:1', 'Actual Negative:0'],  
                          index=['Predict Positive:1', 'Predict Negative:0'])  
  
sns.heatmap(cm_matrix, annot=True, fmt='d', cmap='YlGnBu')
```

Out[208... <Axes: >

```
In [209... plt.show()
```



```
In [210... from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
<=50K	0.94	0.80	0.86	7454
>50K	0.56	0.82	0.67	2314
accuracy			0.81	9768
macro avg	0.75	0.81	0.77	9768
weighted avg	0.85	0.81	0.82	9768

```
In [211... TP = cm[0,0]
TN = cm[1,1]
FP = cm[0,1]
FN = cm[1,0]
```

```
In [212... # print classification accuracy

classification_accuracy = (TP + TN) / float(TP + TN + FP + FN)

print('Classification accuracy : {0:0.4f}'.format(classification_accuracy))
```

Classification accuracy : 0.8062

```
In [213... classification_error = (FP + FN) / float(TP + TN + FP + FN)
print('Classification error : {0:0.4f}'.format(classification_error))
```

Classification error : 0.1938

```
In [214... precision = TP / float(TP + FP)

print('Precision : {0:0.4f}'.format(precision))
```

Precision : 0.8009

```
In [215... recall = TP / float(TP + FN)

print('Recall or Sensitivity : {0:0.4f}'.format(recall))
```

Recall or Sensitivity : 0.9359

```
In [216... true_positive_rate = TP / float(TP + FN)

print('True Positive Rate : {0:0.4f}'.format(true_positive_rate))
```

True Positive Rate : 0.9359

```
In [217... false_positive_rate = FP / float(FP + TN)
```

```
print('False Positive Rate : {0:0.4f}'.format(false_positive_rate))
```

False Positive Rate : 0.4379

```
In [218... specificity = TN / (TN + FP)
print('specificity : {0:0.4f}'.format(specificity))
```

specificity : 0.5621

```
In [219... y_pred_prob = gnb.predict_proba(X_test)[0:10]
y_pred_prob
```

```
Out[219... array([[5.23943398e-02, 9.47605660e-01],
       [9.99967002e-01, 3.29975035e-05],
       [9.99999933e-01, 6.66048938e-08],
       [9.99999998e-01, 1.71025225e-09],
       [9.99767720e-01, 2.32280145e-04],
       [1.02519621e-01, 8.97480379e-01],
       [9.99999956e-01, 4.38548785e-08],
       [9.99305714e-01, 6.94285866e-04],
       [9.83857143e-01, 1.61428566e-02],
       [1.00000000e+00, 3.82008032e-10]])
```

```
In [220... y_pred_prob_df = pd.DataFrame(data=y_pred_prob, columns=['Prob of - <=50K', 'Prob of - >50K'])
y_pred_prob_df
```

Out[220...

	Prob of - <=50K	Prob of - >50K
0	0.052394	9.476057e-01
1	0.999967	3.299750e-05
2	1.000000	6.660489e-08
3	1.000000	1.710252e-09
4	0.999768	2.322801e-04
5	0.102520	8.974804e-01
6	1.000000	4.385488e-08
7	0.999306	6.942859e-04
8	0.983857	1.614286e-02
9	1.000000	3.820080e-10

In [221...

```
gnb.predict_proba(X_test)[0:10, 1]
```

Out[221...

```
array([9.47605660e-01, 3.29975035e-05, 6.66048938e-08, 1.71025225e-09,
       2.32280145e-04, 8.97480379e-01, 4.38548785e-08, 6.94285866e-04,
       1.61428566e-02, 3.82008032e-10])
```

In [222...

```
y_pred1 = gnb.predict_proba(X_test)[: , 1]
```

In [223...

```
# plot histogram of predicted probabilities
```

```
# adjust the font size
```

```
plt.rcParams['font.size'] = 12
```

```
# plot histogram with 10 bins
```

```
plt.hist(y_pred1, bins = 10)
```

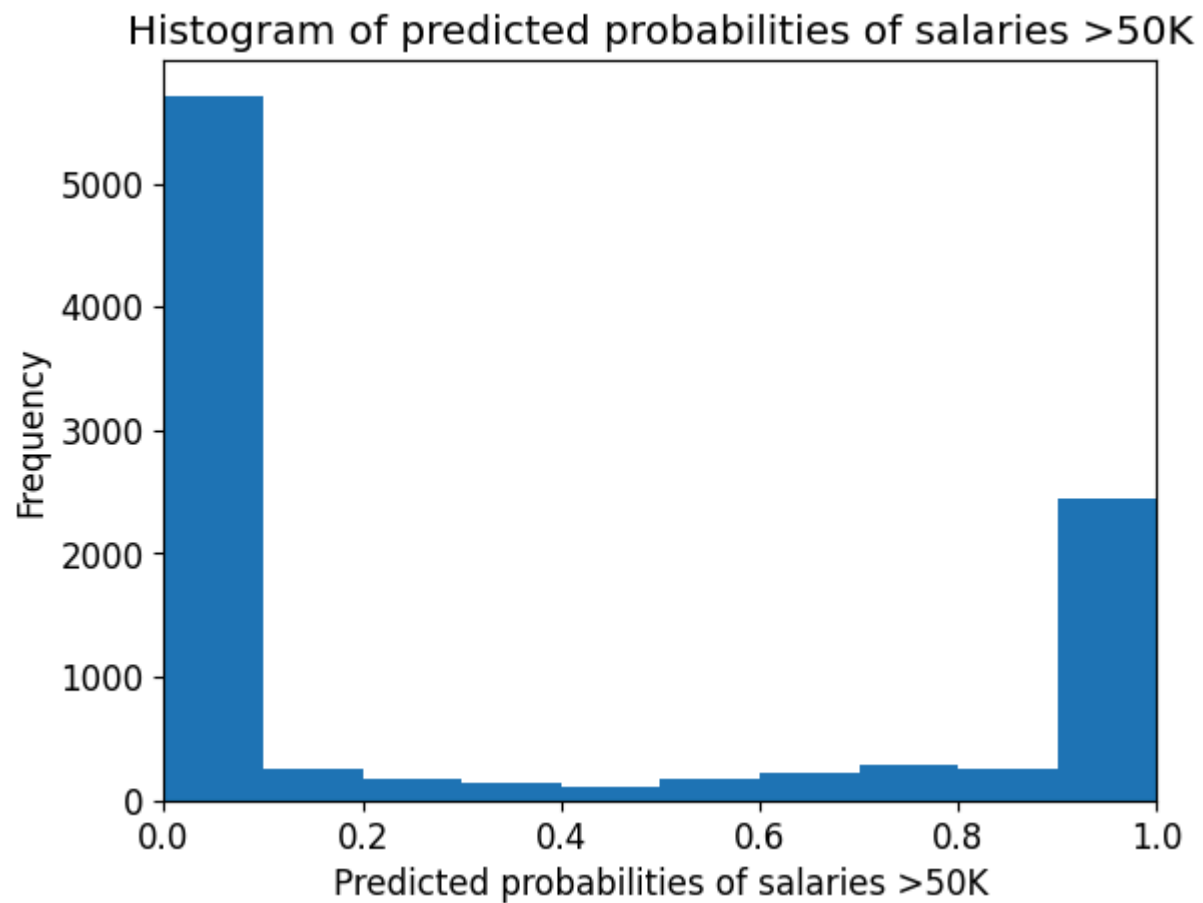
```
# set the title of predicted probabilities
plt.title('Histogram of predicted probabilities of salaries >50K')

# set the x-axis limit
plt.xlim(0,1)

# set the title
plt.xlabel('Predicted probabilities of salaries >50K')
plt.ylabel('Frequency')
```

Out[223... Text(0, 0.5, 'Frequency')

In [224... plt.show()



```
In [225... # plot ROC Curve

from sklearn.metrics import roc_curve

fpr, tpr, thresholds = roc_curve(y_test, y_pred1, pos_label = '>50K')

plt.figure(figsize=(6,4))

plt.plot(fpr, tpr, linewidth=2)

plt.plot([0,1], [0,1], 'k--' )
```

```
plt.rcParams['font.size'] = 12

plt.title('ROC curve for Gaussian Naive Bayes Classifier for Predicting Salaries')

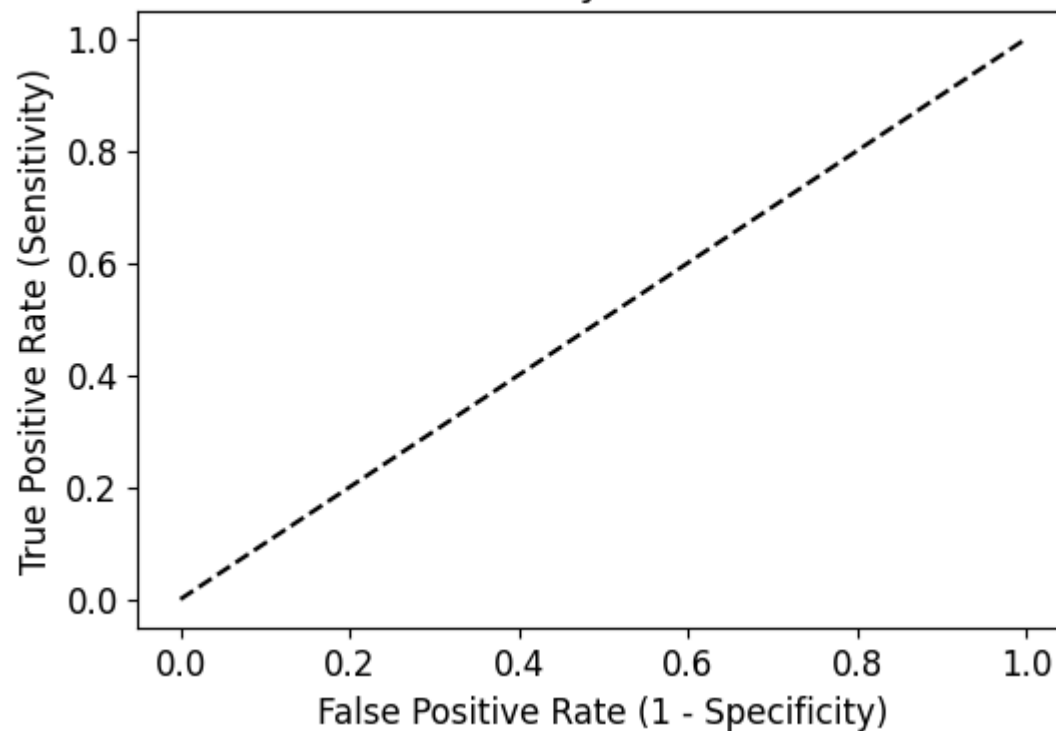
plt.xlabel('False Positive Rate (1 - Specificity)')

plt.ylabel('True Positive Rate (Sensitivity)')
```

Out[225... Text(0, 0.5, 'True Positive Rate (Sensitivity)')

In [226... plt.show()

ROC curve for Gaussian Naive Bayes Classifier for Predicting Salaries



```
In [227... from sklearn.metrics import roc_auc_score

ROC_AUC = roc_auc_score(y_test, y_pred1)
```

```
print('ROC AUC : {:.4f}'.format(ROC_AUC))
```

ROC AUC : 0.8989

In [228... *# calculate cross-validated ROC AUC*

```
from sklearn.model_selection import cross_val_score
```

```
Cross_validated_ROC_AUC = cross_val_score(gnb, X_train, y_train, cv=5, scoring='roc_auc').mean()
```

```
print('Cross validated ROC AUC : {:.4f}'.format(Cross_validated_ROC_AUC))
```

Cross validated ROC AUC : 0.8919

In [229... *# Applying 10-Fold Cross Validation*

```
from sklearn.model_selection import cross_val_score
```

```
scores = cross_val_score(gnb, X_train, y_train, cv = 10, scoring='accuracy')
```

```
print('Cross-validation scores:{}'.format(scores))
```

Cross-validation scores:[0.80570175 0.80570175 0.7977183 0.80386134 0.7819219 0.80912681
0.8135147 0.79464677 0.81000439 0.79859588]

In [230...

```
print('Average cross-validation score: {:.4f}'.format(scores.mean()))
```

Average cross-validation score: 0.8021

In []: